

Your Index Is Fine, Your Query Encoder Is Not: Query Distillation over Frozen Indexes

Dylan Couzon Evgeniya Sukhodolskaya
dylan.couzon@qdrant.com jenny@qdrant.com
Kumar Shivendu Andrey Vasnetsov
kumar.shivendu@qdrant.com andrey@qdrant.com
Qdrant

October 2026

Abstract

A dense document index is often the one component of a retrieval system that cannot be rebuilt: re-encoding is expensive, and every downstream consumer depends on the stored vectors. The query encoder has no such constraint. Query-side distillation trains a cheap student to emit queries in the frozen index’s space, so the index stays and only the query path changes. This paper asks what that substitution costs and what properties of the index decide the cost. Two properties of a frozen index are fixed by the encoder that built it: the width of its vectors and the retrieval quality of its space. We fit query students to 26 public encoder spaces under two student representations, each fitted by a single linear solve with no gradient training (closed form) and ask, first, how much retrieval quality the student recovers and what predicts it. The space’s own retrieval quality predicts the student’s quality only once width is held fixed: standardized effects of +0.69 for quality and −0.83 for width under a frozen-backbone head student (+0.63 and −0.64 under a token-table student), and a two-variable model fitted on ten spaces ranks 16 held-out spaces at Spearman 0.85 and 0.70. The strongest space yields the weakest registered table student. Second, we ask what the student costs at search time over the index’s unchanged HNSW graph: by exact-neighbor recovery its queries fall short of the encoder’s own at equal effort in all 25 measured spaces on two of three corpora, and matching the encoder’s recovery takes a median four times the search budget; by the pre-specified relevance-loss measure the penalty is universal on one corpus and mixed on the other two. The same penalty appears in a jointly trained model: over LightRetriever’s own index, its lookup query path recovers 3.6 points fewer exact neighbors than its full encoder at equal effort and needs four times the search budget to match it. Query-encoding speedups measured alone therefore overstate what a cheap query path saves. Constella, three compatible query encoders over one frozen Stella index, is the worked instance.

1. Introduction

In a production retrieval system the document index is the component that is hardest to change. Re-encoding is the smaller part of the cost and already substantial: the 23.7 million documents of the BEIR-15 collection took a measured 39.9 hours with a 400-million-parameter encoder on one consumer GPU, and production corpora and encoders are larger. The larger part is everything built on the vectors: the graph, its quantization, and every consumer that stores, caches, or joins against them. The query encoder is the opposite: it runs once per request, it can be replaced between requests, and nothing stored depends on it, as long as its output lands in the index’s space.

Query-side distillation makes that condition hold. A student encoder is trained to reproduce the frozen encoder’s query vectors and then searches the frozen document vectors directly [QED, EmbedDistill, LEAF]. Static variants fit a token table instead of a transformer, so a query costs a table lookup and a sum [pyNIFE,

LightRetriever]. The capability is established; so is its ceiling, since methods that co-train the document tower retain more [LightRetriever]. What is not established is what the frozen-index version costs over a given index, and whether the cost can be predicted from the index before anything is built.

Two properties of a frozen index are fixed by the encoder that built it and cannot be changed without rebuilding: the width of its stored vectors and the retrieval quality of the space those vectors come from. We study the question over 26 public encoder spaces. Because the students are fitted by a single linear solve in minutes rather than trained, the same recipe can be applied to every space under one protocol, which turns a one-model demonstration into a comparison across spaces. The same cheapness is what makes the substitution worth considering in production: a student is built in minutes to hours of GPU time, the index is not touched, and the saving recurs on every query (Section 6). Two questions organize the paper.

- **RQ1, retention.** How much retrieval quality can a closed-form query student recover from a frozen index, and do the two encoder-fixed properties, width and the space’s own quality, predict it? The field’s working assumption, which we take as the hypothesis to test, is that a better space yields a better student. RQ1 is a property of the space; the corpus and the graph do not enter it.
- **RQ2, search effort.** Over the index’s unchanged approximate-search graph, how much more search effort does the student’s query path need than the index’s own query path, and is that effort predictable?

Against RQ1, the space’s own retrieval quality predicts the student’s quality only once width is held fixed; width works against the student, and because stronger spaces are usually wider the two effects cancel in a pooled comparison. A two-variable model fitted on ten spaces ranks 16 others at Spearman 0.85 and 0.70 under the two representations, against 0.37 and 0.16 for the space’s own score alone [E24 prospective test pending]. Against RQ2, by exact-neighbor recovery the student’s queries fall short of the encoder’s own at equal HNSW effort in all 25 measured spaces on two of three corpora, with a median recovery multiplier of four; the pre-specified relevance-loss measure agrees on one corpus and is mixed on the other two [E22 and E23 pending]. The penalty is not an artifact of fitting students after the fact: LightRetriever’s jointly trained lookup path pays it over its own index (Section 5.5).

Constella is the worked instance: a frozen Stella index [Stella] with three compatible query paths, Stella’s own encoder, a 34.5-million-parameter transformer student (Nano), and a token-table student (Zero), released with the registered evaluations that gated the release. Section 6 reports it.

Throughout, “registered” marks the project’s pre-registered evaluations, whose decision rules were fixed before any result was observed, and “exploratory” marks analyses added afterwards. The two index-wide screens that answer RQ1 and RQ2 are exploratory; their analyses were written down before scoring, and the prospective test of RQ1 commits its predictions before any student is fitted.

2. Related work

Query-side distillation against a frozen document encoder is established: QED retains 92.5% of a dense retriever’s BEIR score with a two-layer student [QED], EmbedDistill distills an asymmetric student against the teacher’s frozen document encoder [EmbedDistill], and LEAF reports 97.7% retention at 4.7-fold compression [LEAF]. pyNIFE fits a static token table to a frozen teacher [pyNIFE]; LightRetriever trains a lookup query side jointly with a document tower [LightRetriever]; static sentence embeddings, Model2Vec, and NanoVDR are related compact representations [StaticEmb, Model2Vec, NanoVDR]. Methods that train both towers at once reach higher retention: LightRetriever’s lookup query side keeps about 95% of its co-trained document tower’s BEIR score, against 81.4% for Zero over a frozen Stella. That gap is the price of the frozen index: joint training re-encodes every document, which is the operation this paper assumes is not available. Section 5.5 measures LightRetriever’s released model under RQ2’s protocol. None of these works compares students across many teachers under one recipe, which is the comparison RQ1 needs.

That stronger teachers can produce weaker students is known in distillation generally [Cho and Hariharan 2019] and in dense retrieval [PROD]; the explanations offered there concern capacity gaps between teacher and student. Our contribution to that line is a measured competing explanation, the teacher’s width, with a held-out prediction.

Graph-based approximate search over HNSW is sensitive to where queries sit relative to the indexed vectors [HNSW]; out-of-distribution queries are known to need more effort, and graph constructions exist that use query samples to mitigate it [OOD-DiskANN, RoarGraph]. RQ2 measures that effect for queries produced by a student of the same index, which are near the index’s distribution but not on it. Query-aware decoding of binary sketches is the precedent for the precision control in Section 5 [QA-Cos].

3. Experimental setup

3.1 Encoder spaces

The roster is 26 public English embedding encoders that share the 30,522-entry BERT WordPiece vocabulary, a requirement of the table student. Ten were registered before any result; 16 were added as an exploratory expansion under the same recipe and selection order. Each encoder defines one frozen space: its document vectors are frozen and its own query encoder is the reference path. Table A lists them by width.

Table A. The 26 encoder spaces by width (full roster in Appendix B). “Own nDCG@10” is the encoder’s own query path on the six-set suite (Section 3.3); student columns are the outcomes of Section 4.

Width	Spaces (registered)	Own nDCG@10, range	Table student, range	Head student, range
384	9 (0)	0.328 to 0.504	0.291 to 0.358	0.196 to 0.508
768	10 (4)	0.285 to 0.533	0.186 to 0.353	0.059 to 0.325
1024	7 (6)	0.474 to 0.597	0.246 to 0.397	0.156 to 0.268

The roster is related: nine model families, three width levels, and several checkpoints that are revisions of one another. Checkpoint-level intervals in this paper resample checkpoints and do not treat them as independent; family-level sensitivity is reported beside them. One control, arctic-embed-l read out with mean pooling instead of its published CLS pooling, is excluded from every roster statistic.

3.2 Student representations

Two closed-form students are fitted to every index, each producing a 1024-or-narrower query vector in the index’s space.

Table student. A matrix of one row per WordPiece token, fitted by ridge regression from the token counts of each fit query to the index’s query vector for that query, initialized from the index’s own token embeddings and solved by block conjugate gradient with a convergence gate. A query is the sum of its tokens’ rows, each weighted by the square root of the token’s count, then normalized; no transformer runs at query time, so a query costs a tokenizer pass and one vector sum.

Head student. A frozen bge-small backbone, read out as the concatenated masked-mean states of layers 12, 8, and 4 (1,152 dimensions) plus a bias, with a dense ridge head fitted to the index’s query vectors; the output is normalized. The backbone is the feature path of the Nano instance. It is a second representation under the same screening setup, chosen to test whether RQ1’s answer depends on the table.

Both students are floors for a trained student of the same architecture; neither selects the trained Zero or Nano of Section 6.

3.3 Fit data, selection, and scoring

Both students are fitted on the same 337,981 screened fit queries, encoded by each index’s own query path with its published prompt. The ridge penalty is chosen per index on two development forums (CQADupStack physics and programmers) and frozen before any public dataset is scored. Retrieval is scored by exact search, nDCG@10 with the registered self-hit drop, on six public BEIR datasets: NFCorpus, SCIDOCS, SciFact, TREC-COVID, FiQA, and ArguAna. The six-set macro is the outcome; the four sets without disclosed exposure for the Stella instance (the first four) are a sensitivity outcome. Each index is scored once per set.

3.4 Approximate-search protocol

RQ2 uses Qdrant 1.19.1 with HNSW $m=16$, $ef_construct=100$, cosine distance, no quantization. For each encoder space and each of three workloads, FiQA (57,638 documents), SCIDOCS (25,657), and TREC-COVID (171,332), two graphs are built with different seeded insertion orders over the encoder’s document vectors, and ef is swept over 16 to 512 for the encoder’s own queries and both students’ queries, each path measured against its own exact top-10. The pre-specified primary quantity is the relative-nDCG-loss multiplier: with the encoder’s own path at $ef=64$ as the reference, the smallest ef at which the student path reaches the same relative loss, divided by 64, averaged over builds; a space whose student never reaches the reference by $ef=512$ in a build is reported as censored, meaning its multiplier is at least eight and otherwise unknown. The secondary quantity is the same multiplier on tie-aware recovery@10, and the recovery gap at $ef=64$. Parity checks, the tie rule, and the one excluded encoder (tas-b) are in Appendix C.

3.5 The instance

Constella is built around Stella, the `stella_en_400M_v5` encoder, whose 1024-dimensional normalized document vectors are frozen throughout; Zero is a trained table student and Nano a 34.5-million-parameter transformer on the head student’s feature path, both trained against Stella’s query vectors and evaluated on BEIR-15 by exact search (Section 6, Appendix A).

4. RQ1: what the index decides about retention

4.1 Hypothesis and the pooled result

H1. The stronger the index’s own retrieval, the stronger the closed-form student fitted to it; in particular, the strongest index yields the strongest student.

H1 is the heuristic a team would use to choose which space to build a cheap query path for, and it is what the project used before the screen. The roster (Appendix B) contains its pooled refutation: the strongest registered space, gte-large-en-v1.5, yields the weakest registered table student and a below-median head student, and Stella, second in the same width band, yields the strongest table student. Over all 26 spaces, the rank correlation between a space’s own score and its student’s score is +0.09 for the table (95% checkpoint interval -0.39 to $+0.52$) and +0.09 for the head (-0.36 to $+0.51$). The intervals are compatible with a moderate positive association; the strongest-index rule itself is tested directly in Section 4.4.

4.2 Width as the competing explanation

H1’. Conditional on the index’s width, the index’s own quality predicts the student’s quality, and width has a negative effect of its own. H1’ is falsified if the width coefficient’s interval includes zero or if the two-variable model predicts held-out families no better than the index’s score alone. Figure 1 suggests the structure: within each width level student quality rises with index quality, and across levels wider indexes yield weaker students. Table B fits it. The outcome is the student’s absolute six-set score, so that retention, the student’s score divided by the index’s, is a derived quantity rather than the modelled one; the predictors are the space’s own score and $\log_2$ width; coefficients are standardized, so each is the change in student quality, in standard deviations, per standard deviation of the predictor; intervals come from 10,000 resamples of the 26 checkpoints; held-out performance is prediction for each model family from a fit on the other eight (leave-one-family-out).

Table B. Nested models of student quality over 26 indexes (exploratory, analysis pre-specified).

Student	Predictors	Space quality beta	Width beta	In-sample R^2	Leave-one-family- out Spearman
Head	space quality	+0.34 $[-0.16,$ $+0.65]$		0.12	-0.17
Head	space quality, width	+0.69 $[+0.40,$ $+0.97]$	$-0.83 [-1.14,$ $-0.60]$	0.70	$+0.69$

Student	Predictors	Space quality beta	Width beta	In-sample R ²	Leave-one-family- out Spearman
Table	space quality	+0.37 [−0.30, +0.72]		0.14	−0.15
Table	space quality, width	+0.63 [+0.06, +0.89]	−0.64 [−0.95, −0.32]	0.48	+0.40

The space’s quality and its width correlate at +0.61 across the roster, which is why the pooled correlation is near zero; given width, the partial rank correlation between student and index quality is +0.58 for the head and +0.45 for the table. Adding eight family indicators raises in-sample R² to 0.91 and 0.81 and leaves the signs unchanged; those models cannot be validated out of family and are reported only in sample. Under the head representation the width effect is the larger of the two (−0.83 against +0.69); under the table the two are of similar size (−0.64 and +0.63), and the intervals do not separate them.

4.3 Held-out and prospective prediction

The prediction: a two-variable model fitted on part of the roster ranks the rest better than the space’s own score does. The roster’s own history supplies the test. The two-variable model fitted on the ten registered spaces, scored first, ranks the 16 spaces scored later at Spearman 0.85 [0.53, 0.99] for the head and 0.70 [0.20, 0.96] for the table; the space’s own score alone ranks them at 0.37 and 0.16. The hypothesis in Table B was formed after all 26 were seen, so this is a held-out fit rather than a prospective test. [E24 pending: eight further spaces never scored before, with predictions committed by hash before any student was fitted.]

4.4 Corollary: choosing an index before it exists

RQ1 concerns a space that already exists. For a team that can still choose, the same data answer a second question: which rule picks the encoder whose student will be best? Table C scores three rules by the six-set nDCG@10 their pick loses against the best student in the roster.

Table C. Six-set nDCG@10 lost against the best student in the roster, by selection rule.

Rule	Head: pick, regret	Table: pick, regret
Strongest space (H1)	gte-large-en-v1.5, 0.285	gte-large-en-v1.5, 0.152
Two-variable model, leave-one-out pick	arctic-embed-s, 0.249	gte-small, 0.107
Student’s own dev-forum screen	gte-small, 0.035	stella-400M-v5, 0.000

The strongest-space rule loses 0.285 and 0.152. The two-variable model’s leave-one-out pick loses 0.249 and 0.107: the model ranks the roster but does not identify its top. The student’s own development screen, two forums scored with the fitted student, predicts the six-set rank at Spearman 0.84 (head) and 0.88 (table), makes the best or near-best pick, and when added to the two-variable model carries the largest standardized coefficient (+0.71 [+0.36, +0.99] for the head) and raises leave-one-family-out prediction to 0.77 and 0.80.

One caveat governs Section 4: 26 related spaces at three width levels support a hypothesis about what a closed-form student pays for width, not a mechanism; the head’s own backbone is in the roster (bge-small-en-v1.5), and every statistic above changes by less than 0.06 without it.

5. RQ2: what the index decides about search effort

5.1 Hypothesis

H2. Over the index’s unchanged HNSW graph, the student’s query path needs more search effort than the index’s own query path to recover the same fraction of its exact neighbors, because the student’s queries sit differently against the indexed vectors.

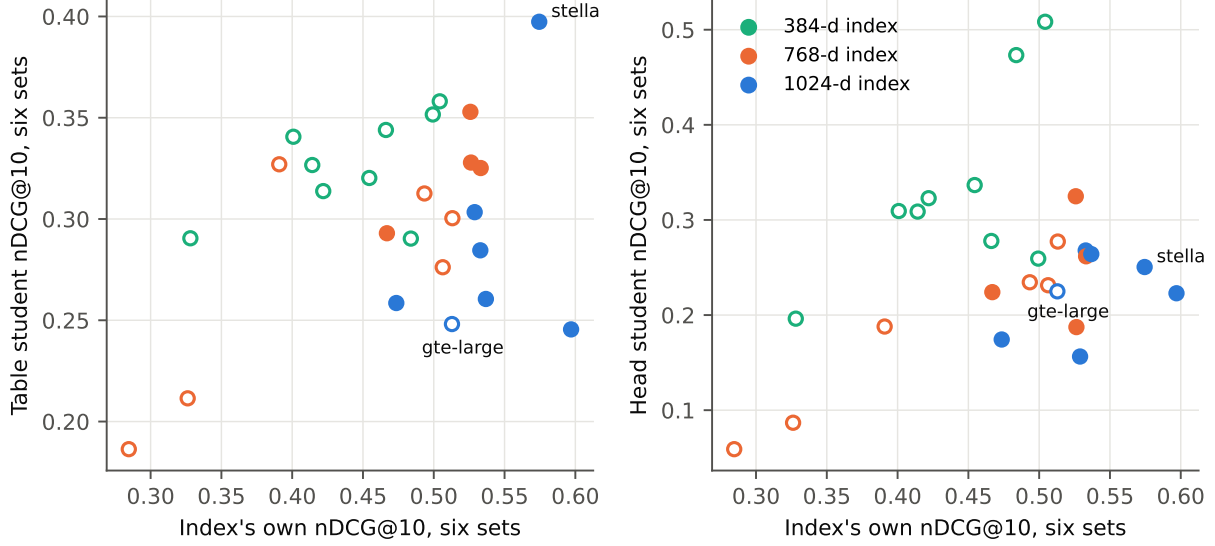


Figure 1: Student quality against the index’s own quality for the table (left) and head (right) students, colored by index width. Filled points are registered indexes.

The first half of H2 is testable per index. The second half is a hypothesis about placement; Section 5.3 tests whether declared placement features predict the size of the effect, which is correlational evidence for it, not an identification of a mechanism.

5.2 Effort across 25 indexes

Table D. Search effort of the table student relative to the index’s own path at $ef=64$, two builds averaged, 25 indexes (exploratory, analysis pre-specified). The primary quantity is the relative-nDCG-loss multiplier; “censored” means at least one build never reached the reference by $ef=512$.

Workload	Documents	Loss multiplier: indexes above 1 / at or below	Censored	Loss multiplier median over reached	Recovery gap at $ef=64$, mean (indexes below 0)	Recovery multiplier median
FiQA	57,638	25 / 0	4	4.0	−2.9 pp (25 of 25)	4.0
SCIDOCS	25,657	17 / 8	8	1.25	−0.8 pp (17 of 25)	2.0
TREC-COVID	171,332	12 / 13	8	0.25	−3.9 pp (25 of 25)	4.0

H2 predicts a multiplier above one in most indexes on every workload. By the primary measure that holds on FiQA (25 of 25, median four over the 21 reached, four censored in at least one build) and on SCIDOCS (17 of 25), and it is mixed on TREC-COVID (12 of 25), where the index’s own path loses only 0.2% of nDCG at $ef=64$ over 50 queries, so a relative-loss threshold there is sensitive to differences smaller than its noise. By the secondary measure it holds on every workload (Table D), and the head student shows the same recovery deficit in 24 and 23 of 25 spaces.

5.3 Is the effort predictable?

H2b predicts that declared features of query placement, computable before a graph is built, rank the spaces by the size of the effect. Two placement summaries, the median cosine of a query to its nearest document and the median gap between its first and tenth scores, are lower for every student path than for its encoder’s, yet across the 25 spaces they do not rank the gap: Spearman 0.00 on FiQA, +0.23 on SCIDOCS, −0.34 on TREC-COVID, intervals spanning zero. [E22 pending: Table E, declared placement features (query-to-document distance, cosine, hubness, top 10 median, effective rank) with clustered intervals and loose family cut

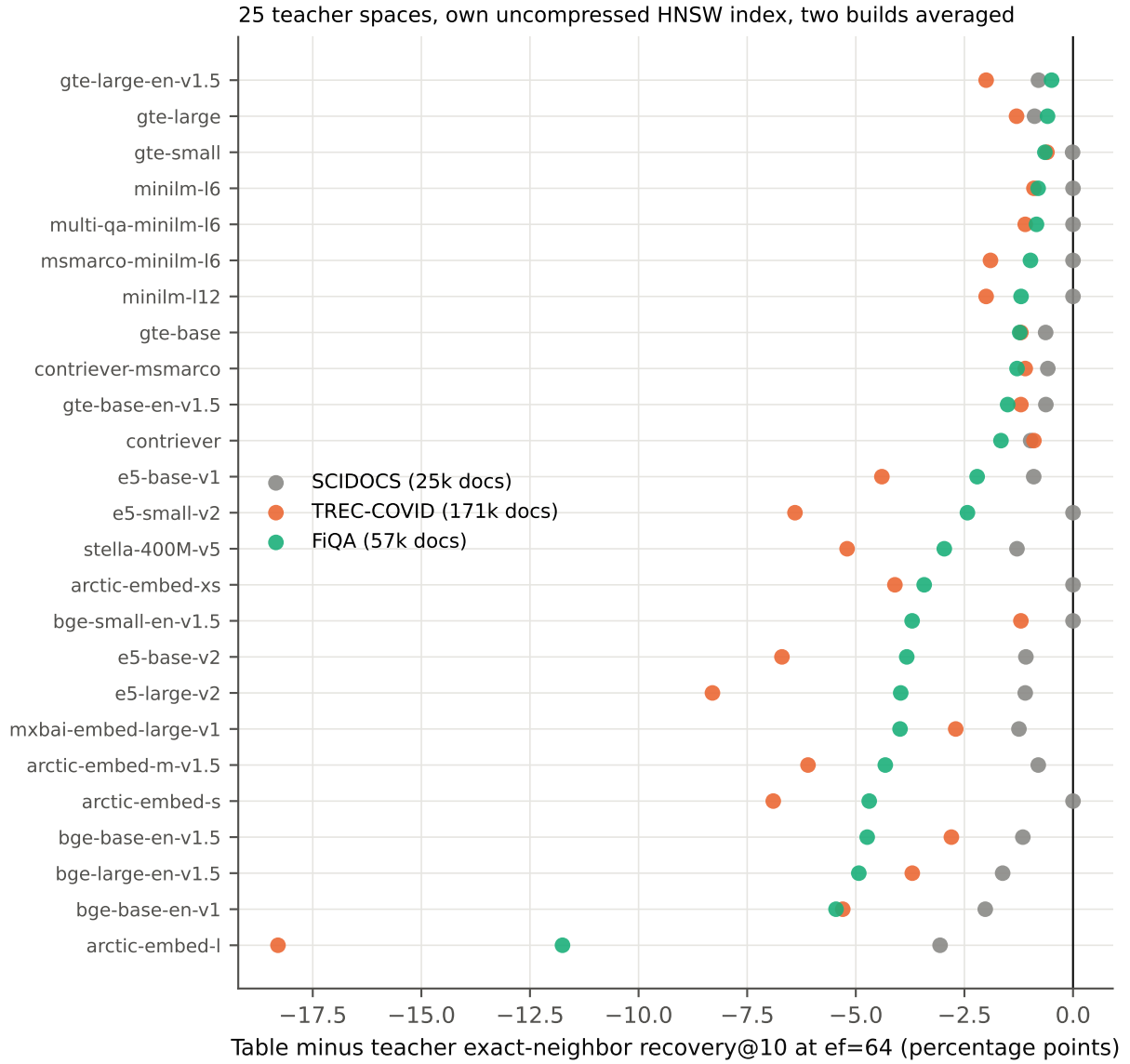


Figure 2: Recovery gap at **ef=64** by index, three workloads, two builds averaged.

prompt and the task prompt as secondary. Predictions and quantities were fixed before encoding (exploratory, pre-specified).

Table E. LightRetriever’s lookup path against its full query path over its own index, two builds averaged.

Workload	Prompt	Exact nDCG@10, lookup / full	Loss multiplier (builds)	Recovery multiplier	Recovery gap at ef=64, query 95% interval
FiQA	web search	0.4065 / 0.4752	3.0 (2, 4)	4.0	−3.6 pp [−4.3, −2.9]
FiQA	task	0.4124 / 0.4766	2.0 (2, 2)	4.0	−2.3 pp [−2.9, −1.8]
SCIDOCS	web search	0.1655 / 0.1925	2.25 (0.5, 4)	4.0	−1.7 pp [−2.1, −1.4]
SCIDOCS	task	0.1818 / 0.2017	censored (1, censored)	2.0	−0.7 pp [−1.0, −0.5]

H2d holds. The lookup path recovers fewer of its own exact neighbors at **ef=64** on both corpora under both prompts, with query intervals clear of zero, and matching the full path’s recovery takes four times the **ef** in every build on FiQA and under the primary prompt on SCIDOCS. The relevance-loss multiplier agrees on FiQA (two to four) and is uninformative on SCIDOCS, where both paths lose under 0.1% of nDCG at **ef=64**. On the pod GPU the lookup path encoded a query in 0.10 ms against 0.90 ms for the full path, a nine-fold gap that the extra **ef** consumes. Neither the model’s published retention nor its encoding-speedup figure is in dispute; the measurement shows that the figure is taken before the step where the cheap path pays. One caveat: one 1.5-billion-parameter model, our reproduction of its dense path, two corpora.

One caveat governs Section 5: a multiplier on **ef** is a multiplier on graph effort, not on latency, and a recovery deficit is a relevance deficit only where the primary measure agrees.

6. The Constella instance

Table 1 gives the three query paths over Stella’s frozen index, by exact search over BEIR-15.

Table 1. Three query paths over one frozen Stella index. Encoding medians: 100 real 5-to-12-word queries, batch one, four CPU threads, Apple M5 Pro, registered protocol.

Query path	Exact nDCG@10	Share of Stella	Encode p50	Assets
Stella query encoder	0.5614	reference	31.6 ms	1669.6 MiB
Nano (34.5M transformer)	0.5081	90.5%	2.25 ms	132.3 MiB
Zero (token table)	0.4572	81.4%	0.044 ms	90.1 MiB

The three paths differ in what they compute, and therefore in where each loses. Zero reads one learned vector per token and sums them, so it cannot use word order or context: two queries with the same tokens in a different order get the same vector. Its retention is lowest on topical and scientific sets (0.67 on TREC-COVID and FiQA) and highest where the query is a short entity or a near-duplicate of a document (0.95 on Quora, 0.96 on climate-fever), and in an internal domain-jargon workload it fell below BM25. Nano runs three layers of a small transformer over the query, so it sees context at about a fiftieth of Stella’s cost; its retention is flat across datasets (0.85 to 0.99 except FEVER, which was in Zero’s training pool and not in Nano’s), and on BEIR-15 it is below two small models that use their own indexes (Appendix A). Stella’s own encoder is the reference and costs 31.6 ms per query. Per-dataset retention is in Appendix A.

Because all three were verified against the same populated collections with no rebuild between encoder batches, a request can use any of them, and an application can match the path to the moment: a table

lookup while the user is typing, the small transformer when typing pauses, and the full encoder on submit. This is a capability of the design, not a measured result; the prefix experiment in Appendix C shows the dense paths retain similar fractions of their full-query quality on truncated queries, and no cascade was timed end to end. Three attempts to specialize a student to one workload over the same index did not produce an improved encoder (Section 7).

What the substitution costs to build and what it saves. Building a table student for an existing index took four to seven minutes of A100 time per encoder in the screen, target encoding included; the head student’s features took 43 seconds for the fit list and its fit 14 seconds per encoder; Nano, a trained transformer student, took 57.3 A100 hours for its final optimization, about \$95 at the recorded rental rate, excluding data preparation, recipe search, and failed runs (Appendix B). None of these touched the index: no document was re-encoded and no collection was rebuilt, so the new path was added to the live collections and verified against them without downtime. On the query side the saving recurs. Table 2 converts the measured medians into CPU time per million queries on the measurement laptop, encoding at batch one with four threads plus search at each path’s within-1% setting on the uncompressed million-passage collection; it is arithmetic on registered medians, not a throughput measurement, and a production system would see different absolute times.

Table 2. CPU time per million queries on the measurement laptop, from the medians in Table 1 and Appendix C.

Query path	Encoding	Search, within 1% of own exact	Total	Share of Stella
Stella query encoder	8.78 h	0.43 h (ef =128)	9.2 h	reference
Nano	0.63 h	0.61 h (ef =256)	1.2 h	13%
Zero	0.01 h	0.99 h (ef =512)	1.0 h	11%

After search, Nano and Zero cost about the same per million queries on this collection, because Zero’s encoding saving is spent on the extra **ef** that RQ2 measures; both are about a ninth of the full encoder’s cost, at 90.5% and 81.4% of its quality. Section 5.5 shows the same accounting applies to a jointly trained lookup path over its own index.

The instance also shows RQ2’s cost in time. On the uncompressed million-passage diagnostic, search time at a fixed **ef** is the same for all three paths within 5%, and the paths differ in the **ef** they need: to stay within 1% of their own exact score, Stella needs **ef**=128, Nano 256, and Zero 512, with search medians of 1.53, 2.18, and 3.58 ms. On one binary-quantized collection the fastest setting within 1% gives Zero 0.6116 at 1.11 ms, Nano 0.6893 at 2.48 ms, and Stella 0.7226 at 21.3 ms: on that collection’s query sample the encoding gap between Zero and Nano is 63-fold (51-fold on Table 1’s sample) and becomes 2.2-fold after search, at different quality levels. Figure 3 gives the registered sweep. Lexical fusion, per-query routing, and a query-precision control over the same index are reported in Appendices C and D.

7. Limitations

RQ1 rests on two closed-form students over 26 related indexes at three width levels; the students are floors for trained students of the same architecture, the screen does not select the trained Zero or Nano, checkpoint intervals do not cover training variance, and width covaries with family and quality. RQ2 rests on one engine and one graph configuration, three workloads that confound corpus size with domain, and 50 queries on TREC-COVID; its timing ran on a shared machine and is not a latency claim. The instance’s two students have different training pools, including different FEVER exposure, and Stella discloses training exposure on ArguAna, FiQA, and FEVER. Query intervals condition on the trained models. The native magnitude-preserving query setting was not tested. The LightRetriever measurement covers one released 1.5-billion-parameter model, our reproduction of its dense path, and two corpora. Three bounded attempts to specialize a student to one internal workload over the same index produced no improved encoder, so per-workload students remain a possibility of the design, not a result.

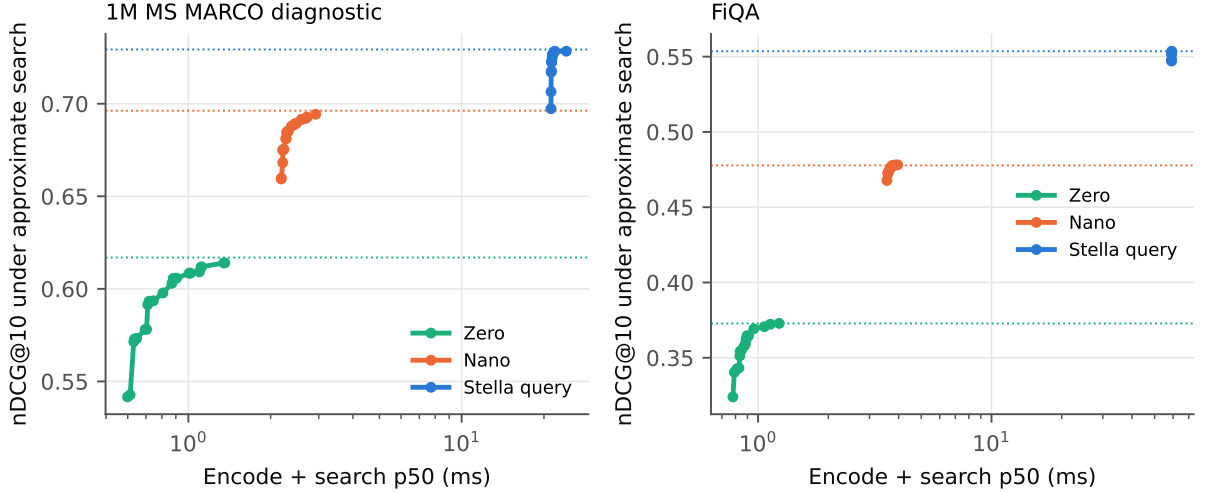


Figure 3: Quality under approximate search against encoding plus search time, registered sweep. Dotted lines are each path’s exact score.

Artifact availability

The companion repository holds inference examples, build configurations, methods with their dated amendments, immutable result receipts, per-dataset tables, and the full research history including negative results. The released artifacts are Constella Zero, Constella Nano, and the Stella document encoder on the Hugging Face Hub. All released paths run in fp32 through the published loader; a candidate fp16 export passed CPU parity and failed on CUDA (minimum cosine 0.662), which is why qualification runs through the actual loader and device. Two inputs are identified by hash but not yet archived in the repository: the cleaned 337,981-query fit list and the sampled one-million-passage identifier list.

Appendix A. Registered evaluations of the instance

Serving configuration. Stella is pinned to revision ffeb2b7e, with its s2p_query prompt, fp32 encoding, normalized output, fp16 stored document vectors, and cosine similarity; maximum sequence length 512. Nano uses bge-small layers 12, 8, and 4 concatenated to 1152 dimensions before projection; batches mix 75% queries and 25% documents. Zero’s training pool includes FEVER-train and HotpotQA; Nano excludes FEVER. MS MARCO is licensed for non-commercial use, so it was validation-only in our distillation: it supplied no training targets, negatives, or generation seeds, although Nano’s pretrained base has prior exposure to it.

Figure A1. Retention per dataset. Datasets with disclosed Stella training exposure are ArguAna, FiQA, and FEVER; the four originally held-out sets are FEVER, DBpedia-entity, and the CQADupStack android and english forums.

Partitions. The six-set suite is NFCorpus, SCIDOCS, SciFact, TREC-COVID, FiQA, and ArguAna. “Clean-four” means the first four, which have no disclosed Stella overlap; it is not a causal contamination estimate. Four datasets were held out and evaluated once: FEVER, DBpedia-entity, and the CQADupStack android and english forums. Two other forums, physics and programmers, select table penalties and routing settings and were reused during model construction.

Reference systems. bge-small-en-v1.5 and LEAF were the project’s selected targets, not a state-of-the-art frontier. They use their own document representations. On BEIR-15 they score 0.5171 and 0.5402 against Nano’s 0.5081, at 2.57 and 1.39 ms median encoding; BM25 scores 0.4006. Nano is below both on the broad average even though it passed some of the registered contrasts below.

Nano’s registered contrasts used a fixed sequence with one-sided 2.5% lower bounds. “Established”

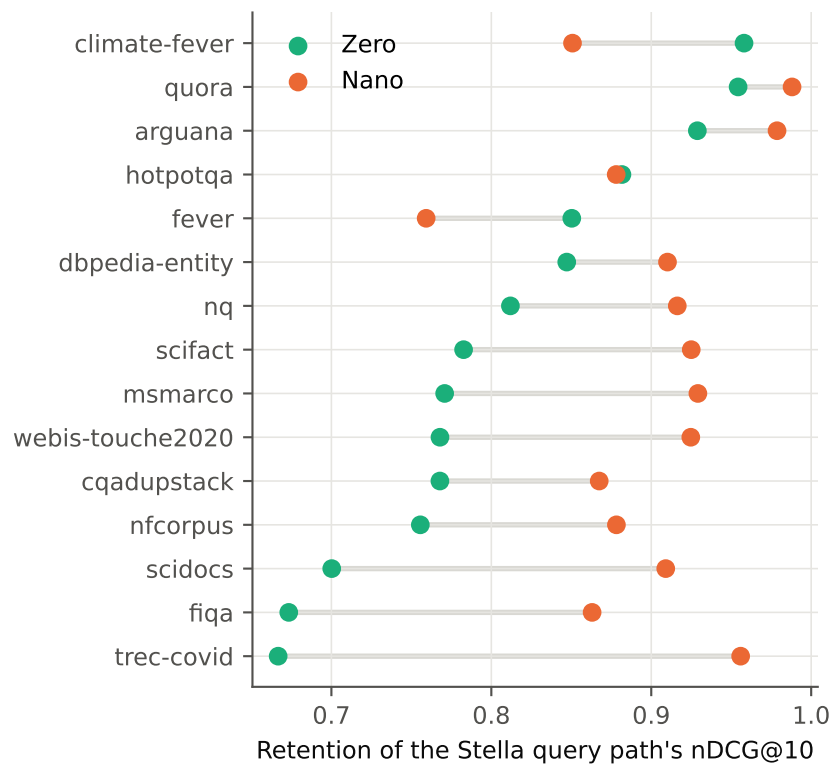


Figure 4: Per-dataset retention of the Stella query path by Zero and Nano over 15 BEIR datasets

means the bound cleared zero; “unresolved” means it did not; “descriptive” rows were registered as reports, not tests. Intervals resample queries with the models fixed.

Contrast	Partition	nDCG@10 difference	Outcome
Nano – bge-small	clean-four	+0.0176; lower bound +0.0037	established
Nano – bge-small	all six	+0.0274; lower bound +0.0173	established
Nano – LEAF	all six	+0.0162; lower bound +0.0065	established
Nano – LEAF	clean-four	−0.0011; lower bound −0.0145	unresolved
Nano – bge-small	held-out NDO-3, dataset weighted	+0.0032 [−0.0069, +0.0134]	descriptive
Nano – bge-small	held-out NDO-3, query pooled	−0.0121 [−0.0219, −0.0024]	descriptive
Nano – LEAF	held-out NDO-3	−0.0389 [−0.0488, −0.0290]	descriptive

NDO-3 is the registered held-out aggregate: DBpedia-entity weighted one half, the android and english forums one quarter each, FEVER excluded as the double-exposure sensitivity row.

Zero’s registered contrasts used a Holm correction across the family. Zero missed its dense bar and did not confirmatorily beat BM25.

Contrast, all six	nDCG@10 difference	Raw 95% interval	Outcome
Zero – LightRetriever dense	−0.0243	[−0.0405, −0.0086]	below the 0.4583 bar
Zero – BM25	+0.0165	[+0.0017, +0.0311]	unresolved; p=.0149 fails Holm .0083
Zero + BM25 (convex) – OpenSearch	+0.0043	[−0.0063, +0.0151]	unresolved

The registered clean-four sensitivity is descriptive: −0.0443 [−0.0675, −0.0212], −0.0311 [−0.0517, −0.0109], and −0.0107 [−0.0262, +0.0043]. Unresolved superiority does not establish equivalence. The confirmatory fusion operator was convex score fusion with development-fitted weight 0.8 at depth 1000; the later DBSF-at-100 recommendation is a deployability choice, not a new confirmation. On the development components DBSF minus convex is +0.0035 at 10 candidates, −0.0020 at 50, −0.0063 at 100, and −0.0146 at 1000.

The held-out four, copied from their one-shot published aggregate:

System	FEVER	DBpedia	CQADup android	CQADup english
Nano	0.6231	0.4190	0.4774	0.4298
Zero	0.6978	0.3900	0.4431	0.3690
Stella	0.8207	0.4603	0.5275	0.5183
bge-small	0.8662	0.4003	0.4761	0.4558
LEAF	0.8671	0.4520	0.5260	0.4707
BM25	0.5036	0.3045	0.3952	0.3481
Zero + BM25	0.7559	0.4030	0.4629	0.4016
Nano + BM25	0.6879	0.4296	0.4750	0.4323

Appendix B. Construction details

The roster. The 26 encoder spaces of Section 3.1, by width and own six-set score.

Encoder (frozen space)	Width	Pooling	Own nDCG@10	Table student	Head student	Roster
bge-small-en-v1.5	384	cls	0.5042	0.3581	0.5084	exploratory
arctic-embed-s	384	cls	0.4993	0.3516	0.2594	exploratory
gte-small	384	mean	0.4838	0.2903	0.4733	exploratory
arctic-embed-xs	384	cls	0.4662	0.3440	0.2779	exploratory
e5-small-v2	384	mean	0.4544	0.3203	0.3367	exploratory
minilm-l12	384	mean	0.4219	0.3138	0.3229	exploratory
minilm-l6	384	mean	0.4142	0.3267	0.3088	exploratory
multi-qa-minilm-l6	384	mean	0.4008	0.3406	0.3094	exploratory
msmarco-minilm-l6	384	mean	0.3281	0.2905	0.1962	exploratory
gte-base-en-v1.5	768	cls	0.5331	0.3252	0.2618	registered
arctic-embed-m-v1.5	768	cls	0.5263	0.3279	0.1874	registered
bge-base-en-v1.5	768	cls	0.5259	0.3529	0.3249	registered
bge-base-en-v1	768	cls	0.5131	0.3004	0.2773	exploratory
gte-base	768	mean	0.5063	0.2762	0.2314	exploratory
e5-base-v1	768	mean	0.4934	0.3126	0.2345	exploratory
e5-base-v2	768	mean	0.4669	0.2930	0.2241	registered
contriever-msmarco	768	mean	0.3909	0.3270	0.1879	exploratory
tas-b	768	cls	0.3262	0.2115	0.0868	exploratory
contriever	768	mean	0.2846	0.1864	0.0590	exploratory
gte-large-en-v1.5	1024	cls	0.5970	0.2455	0.2229	registered
stella-400M-v5	1024	mean	0.5745	0.3974	0.2505	registered
mxbai-embed-large-v1	1024	cls	0.5368	0.2605	0.2641	registered
bge-large-en-v1.5	1024	cls	0.5329	0.2845	0.2679	registered
arctic-embed-l	1024	cls	0.5289	0.3034	0.1564	registered
gte-large	1024	mean	0.5129	0.2481	0.2249	exploratory
e5-large-v2	1024	mean	0.4735	0.2585	0.1743	registered

Table student. Ridge regression from WordPiece token counts to the index’s query vectors, initialized from the index’s own token embeddings, solved by block conjugate gradient with a convergence gate; penalty grid 1e-4 to 1e-1 with one decade of extension at an edge, chosen on the two development forums and frozen before six-set scoring. The fit list was cleaned after an earlier version had 1.31% held-out-query overlap. One checkpoint failed the convergence gate and is excluded. Registered checkpoint exposure is documented;

the exploratory expansion was not individually audited.

Head student. Frozen bge-small at the Nano build’s pinned revision; masked-mean-pooled hidden states of layers 12, 8, and 4 concatenated with a bias column; dense ridge with penalty scaled by the feature Gram trace, same grid, same selection order, same freeze; output normalized. bge-small-en-v1.5 is also an index in the roster and shares the backbone, and gte-small is nearly its linear image (retention 1.008 and 0.978); correlations without the backbone are +0.09 against the index and +0.46 against the table.

Correlation rosters. Registered ten: Spearman head-versus-index +0.18 (−0.56 to +0.67), head-versus-table +0.24 (−0.49 to +0.74), head development-versus-public +0.71 (+0.12 to +0.96). Leave-one-family-out values range from +0.44 to +0.60 for head-versus-table and −0.12 to +0.27 for head-versus-index over the 26. Within width bands, head-versus-index Spearman is +0.47 [−0.47, +1.00] at 384 (n=9), +0.67 [−0.03, +1.00] at 768 (n=10), and +0.36 [−0.76, +0.87] at 1024 (n=7).

Fidelity is not retrieval. Tables fitted to two e5 checkpoints match their index’s query vectors at mean cosine 0.90 and 0.89 against 0.78 for Stella’s table, yet retrieve worse; three earlier training screens reduced their losses without improving ranking. Three vector-error diagnostics did not rank table quality either.

Costs. Table fitting four to seven minutes per registered index on an A100, including query encoding except cached Stella targets; head features 43 seconds for the fit list, dev grid 14 seconds per index, six-set scoring about one minute per index. Nano’s final optimization ran 57.3 hours on one A100 for 199,999,721 example presentations, about \$95 at the recorded rental rate, excluding target preparation, data generation, recipe search, failed runs, and evaluation. Zero’s retraining was estimated at 20 minutes with prepared targets, plus 8 to 12 hours to encode targets for a new index.

Appendix C. Search and precision controls

Instance sweep. Qdrant 1.19.1, HNSW m=16, ef_construct=100, indexing threshold 1 KB so no segment is served by a plain scan, ef in 16 to 512, oversampling 1, 2, and 4 under compression, exact parity against NumPy on 200 queries per path. Timing is warmed and sequential with encoding and search measured in separate phases. At ef=16 the uncompressed million-passage losses are 4.4%, 7.0%, and 16.3% for Stella, Nano, and Zero; under binary quantization without oversampling 6.1%, 9.2%, and 19.8%.

Cross-index sweep. The same engine and graph parameters, uncompressed, two builds per index and workload differing in a seeded insertion order, 16 to 512 ef, limit 11 with the self-hit drop, tie-aware exact parity on 200 queries per path. Four amendments were recorded during the run, before any index’s rows were analysed: SCIDOCS added as a third workload because TREC-COVID has 50 queries; the tie-aware measure; a 0.995 sanity gate on parity with all values reported; and the exclusion rule that removed tas-b (SCIDOCS head parity 0.994). Stella’s own-path rows agree with the instance sweep at every ef. Loss-based censoring: four indexes on FiQA, eight on SCIDOCS, eight on TREC-COVID; recovery-based censoring: one per workload.

Truncated queries (registered). On synthetic prefixes of SciFact, NFCorpus, and FiQA queries, the three dense paths retain descriptively similar shares of their own full-query nDCG@10 at each prefix length; this is not an equivalence test and no search-as-you-type session was measured.

Native binary pilot and replication. On one fixed binary collection per workload with 192 deterministically selected queries, binary scoring with rescoring at 1x oversampling loses 11.7, 5.1, and 2.3 percentage points of exact-neighbor recovery for Zero, Nano, and Stella on FiQA at ef=64; the million-passage replication loses 3.9, 3.1, and 2.2, and the extra Zero-versus-Nano interval includes zero. Native rescoring also changes cross-segment candidate merging.

Graph-free precision control. Same queries, vectors, and targets; all document sign codes scored exhaustively at global candidate budgets 10, 40, and 100; expected coverage averages uniform inclusion at tied boundaries.

Workload	Path	Sign query	Full query	Gain, points
FiQA	Zero	0.8217	0.9297	+10.80
FiQA	Nano	0.9137	0.9771	+6.33
FiQA	Stella	0.9638	0.9922	+2.84
1M diagnostic	Zero	0.9010	0.9677	+6.67
1M diagnostic	Nano	0.9484	0.9833	+3.49
1M diagnostic	Stella	0.9685	0.9932	+2.47

The paired extra Zero gains over Nano are 4.46 points on FiQA (95% query interval 2.53 to 6.43) and 3.17 on the diagnostic (1.58 to 4.75). Zero starts with more misses and does not repair a larger fraction of them. Full query values are not native scalar8 encoding, and global candidate counts are not per-segment oversampling.

Appendix D. Fusion and routing over the frozen index

Lexical fusion. Fusing Zero with BM25 raises its BEIR-15 macro score from 0.4572 to 0.4933, 87.9% of Stella; Nano plus BM25 reaches 0.5110. The sign varies by workload: Zero on HotpotQA rises from 0.6127 to 0.7015, Nano on MS MARCO falls from 0.4063 to 0.3699. On the million-passage diagnostic a dense-plus-sparse collection takes 4.71 ms per query including Zero encoding against 1.11 ms for the dense binary example, in different configurations, not an isolated fusion overhead. The scores use distribution-based score fusion over 100 candidates per branch with a bm25s lexical branch [DBSF]; other lexical implementations need their own comparison because their score distributions enter the normalization. The operator ordering changes with candidate depth (Appendix A).

Routing and blending (registered). On 12 datasets, an oracle choosing Zero or Nano per query with relevance labels scores 0.5473 against 0.5161 for always-Nano, and matches always-Nano by sending the most beneficial 15% of queries to Nano. Query length, subwords per word, and the pooled vector norm are weak selectors (0.002 to 0.005 over random routing at matched Nano share); Zero’s post-search score margin recovers about a quarter of the oracle gain over random routing on six datasets and costs a second search for escalated queries. A development-selected 30% Zero contribution blended into Nano’s vector lowers Nano’s six-set score by 0.0051 (95% paired interval -0.0092 to -0.0011), consistent with how hard retrieval-strategy selection and query performance prediction remain [Arabzadeh et al., QPP].

References

- [QED] Yuxuan Wang and Hong Lyu. Query Encoder Distillation via Embedding Alignment is a Strong Baseline Method to Boost Dense Retriever Online Efficiency. SustaiNLP Workshop at ACL, 2023. [arXiv:2306.11550](#).
- [EmbedDistill] Seungyeon Kim, Ankit Singh Rawat, Manzil Zaheer, et al. EmbedDistill: A Geometric Knowledge Distillation for Information Retrieval. 2023. [arXiv:2301.12005](#).
- [LEAF] Robin Vujanic and Thomas Rueckstiess. LEAF: Knowledge Distillation of Text Embedding Models with Teacher-Aligned Representations. 2025, revised 2026. [arXiv:2509.12539](#).
- [NanoVDR] Zhuchenyang Liu, Yao Zhang, and Yu Xiao. NanoVDR: Distilling a 2B Vision-Language Retriever into a 70M Text-Only Encoder for Visual Document Retrieval. 2026. [arXiv:2603.12824](#).
- [LightRetriever] Guangyuan Ma, Yongliang Ma, Xuanrui Gou, Zhenpeng Su, Ming Zhou, and Songlin Hu. LightRetriever: A LLM-based Text Retrieval Architecture with Extremely Faster Query Inference. ICLR, 2026. [arXiv:2505.12260](#).
- [pyNIFE] Stephan Tulkens. pyNIFE: Nearly Inference Free Embeddings in Python. Software, 2025. [Repository](#).
- [Model2Vec] MinishLab. Model2Vec: Fast State-of-the-Art Static Embeddings. Software. [Repository](#).
- [StaticEmb] Tom Aarsen. Train 400x faster Static Embedding Models with Sentence Transformers. Hugging Face, 15 January 2025. [Blog post](#).
- [Cho and Hariharan 2019] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation. ICCV, 2019. [Proceedings](#).

- [PROD] Zhenghao Lin, Yeyun Gong, Xiao Liu, et al. PROD: Progressive Distillation for Dense Retrieval. WWW, 2023. [arXiv:2209.13335](#).
- [HNSW] Yury A. Malkov and Dmitry A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE TPAMI 42(4), 2020. [arXiv:1603.09320](#).
- [OOD-DiskANN] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. 2022. [arXiv:2211.12850](#).
- [RoarGraph] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. PVLDB 17(11): 2735-2749, 2024. [Paper](#).
- [QA-Cos] Daehun Nyang. Beyond Hamming: Query-Aware Decoding of Binary Cosine Sketches. ICML 2026, PMLR 306: 94126-94143. [Proceedings](#).
- [Arabzadeh et al.] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. CIKM, 2021. [arXiv:2109.10739](#).
- [QPP] Guglielmo Faggioli, Thibault Formal, Stefano Marchesin, Stéphane Clinchant, Nicola Ferro, and Benjamin Piwowarski. Query Performance Prediction for Neural IR: Are We There Yet? 2023. [arXiv:2302.09947](#).
- [DBSF] Qdrant contributors. Hybrid queries: distribution-based score fusion. Qdrant documentation. [Page](#).
- [Qdrant 1.19.1] Qdrant contributors. Binary query encoding and vector-index search implementation, v1.19.1. [Tagged source](#).
- [Stella] Stella-en-400M-v5. Model card for NovaSearch/stella_en_400M_v5. [Model page](#).
- [BEIR] BEIR contributors. BEIR retrieval benchmark, datasets, and evaluation software. [Repository](#).